STC89C52 Microcontroller Multi-Process Simulator

Multi-Process System Architecture & Oral Defense Engineering Manual

Complete System Engineering Manual & Oral Examination Defense Guide

Document Overview

- **Project Name:** STC89C52 8-bit Microcontroller Multi-Process Simulator & IPC Framework
- **Team Name:** Bondaas
- **Institution:** Department of Computer Science & Engineering
- **Course Milestone:** Week 4 -- Multi-Process System Decomposition & POSIX Inter-Process Communication
- **Programming Language:** Java (Standard JDK 11+, Zero External Dependencies)
- **Architecture:** Decoupled 3-Process Microkernel Topology via POSIX TCP Socket IPC

1. Executive System Architecture & Microcontroller Foundation

1.1 The STC89C52 Microcontroller Hardware Model

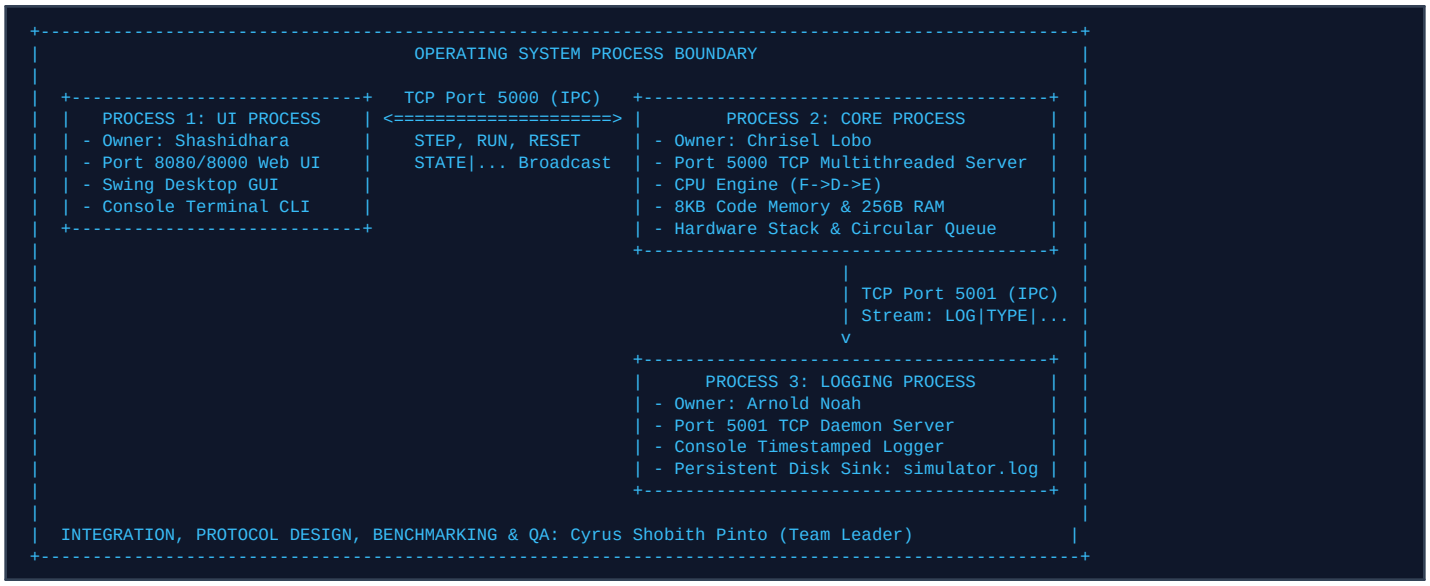
The simulator models the core architecture of the **STC89C52**, an enhanced industry-standard 8051-family 8-bit microcontroller.

1. **Harvard Architecture:** Strict physical and logical separation between **Program Code Memory (ROM)** and **Data Memory (RAM)**. Code and data occupy independent address spaces and cannot overwrite each other via standard instructions.
2. **Program Code Memory:** 8 Kilobytes (8192 bytes), addressed linearly from `0x0000` to `0x1FFF`. Program instructions are fetched sequentially via a 16-bit **Program Counter (PC)**.
3. **Internal Data RAM:** 256 bytes ($0x00$ to $0xFF$).
- `0x00-0x07`: Register Bank 0 (`R0` through `R7`).
- `0x08-0x0F`: Register Bank 1 (`R0` through `R7`).
- `0x10-0x17`: Register Bank 2 (`R0` through `R7`).
- `0x18-0x1F`: Register Bank 3 (`R0` through `R7`).
- `0x20-0x2F`: 16 bytes of bit-addressable memory (128 individual bit locations).
- `0x30-0x7F`: 80 bytes of general-purpose scratchpad RAM.
- `0x80-0xFF`: Upper 128 bytes containing Special Function Registers (SFRs: `ACC`, `B`, `PSW`, `SP`, `DPL`, `DPH`, `P0`-`P3`).
4. **Primary Working Registers:**
- **Accumulator (`A` / `ACC`):** Primary 8-bit register for all mathematical, logical, and data movement operations.
- **B Register (`B`):** Secondary 8-bit math register used as destination/operand during multiplication and division.
- **General Registers (`R0`-`R7`):** 8 general-purpose working registers per bank. The active bank is selected by bits `RS1` and `RS0` in the Program Status Word (`PSW`).
5. **Program Status Word (`PSW`):**
- Bit 7: Carry Flag (`CY`) -- Set if addition exceeds 255 or subtraction borrows.
- Bit 6: Auxiliary Carry Flag (`AC`) -- Set if addition generates a carry out of bit 3 into bit 4 (lower nibble), required for Binary Coded Decimal (`DA A`).
- Bit 5: Flag 0 (`F0`) -- General-purpose user flag.
- Bits 4-3: Register Bank Select (`RS1`, `RS0`) -- Selects active bank 0, 1, 2, or 3.
- Bit 2: Overflow Flag (`OV`) -- Set when signed two's complement arithmetic overflows (-128 to +127).
- Bit 1: Reserved.
- Bit 0: Parity Flag (`P`) -- Automatically computed by hardware; set to `1` if `ACC` contains an odd number of set bits (odd parity), `0` for even parity.
6. **Hardware Stack:**
- Managed by the 8-bit **Stack Pointer (`SP`)**.
- On hardware reset, `SP` defaults to `0x07`.
- The STC89C52 stack is **pre-increment on PUSH** (increments SP, then writes) and **post-decrement on POP** (reads value, then decrements SP).

- Because `SP` starts at `0x07`, the very first pushed item lands at RAM address `0x08` (the start of Bank 1), safely avoiding corruption of active Bank 0 registers (`0x00`-`0x07`).

1.2 System Decomposition: Why Multi-Process?

In Weeks 1 to 3, the simulator was built as a monolithic single-process program where the UI, CPU, and logging mechanisms shared a single JVM heap. Week 4 transforms this into an **enterprise-grade, fault-tolerant system architecture**:



Engineering Advantages of Process Isolation:

1. **Fault Isolation:** A GUI rendering crash or Web browser disconnect cannot crash the CPU core or corrupt microcontroller memory.
2. **Crash-Resilient Audit Logging:** If `CoreProcess` encounters an illegal instruction or memory segmentation fault, `LoggerProcess` remains alive in its own operating system address space and preserves the audit trail to disk.
3. **Decoupled Development:** Each team member owns, tests, and compiles their process completely independently.
4. **Standard Embedded Tooling Analogy:** This mirrors professional embedded systems environments such as GDB (GDB Server runs on target, GDB GUI client runs on host, connected via sockets).

1.3 The POSIX Inter-Process Communication (IPC) Protocol

Mechanism Selection Justification:

The team evaluated POSIX IPC alternatives:

- **Shared Memory (`shm_open`):** Ultra-fast, but non-portable in pure Java without native C JNI/JNA wrappers, and introduces race conditions requiring complex POSIX semaphores.
- **POSIX Named Pipes / FIFOs:** Platform-specific, unidirectional, and difficult to manage cleanly across Windows and POSIX.
- **TCP Loopback Sockets (`127.0.0.1`):** Selected mechanism. Compliant with POSIX.1-2017 (Berkeley Sockets API). It provides a reliable, ordered, bi-directional, packet-delimited byte stream. Works identically across Windows, Linux, and macOS using standard Java `java.net.Socket`.

Strict Packet Syntax:

All socket packets are plain-text UTF-8 strings terminated by a newline (`\n`).

1. **UI to Core Directives (Port 5000):**

- `STEP` -- Advance CPU by one single instruction (execute Fetch -> Decode -> Execute).
- `RUN` -- Execute continuously in a loop until a `HALT` opcode (`0xFF`) or unknown opcode is reached.
- `STOP` -- Asynchronously preempt execution and halt the CPU immediately.
- `RESET` -- Hardware reset: resets PC to `0x0000`, clears registers to `0`, resets Stack Pointer to `0x07`, clears queue, reloads active code memory.
- `LOAD <hex>` -- Ingests a hexadecimal string of machine code bytes, flashes code memory starting at `0x0000`, appends `0xFF` if omitted, and performs a reset.
- `STATE` -- Queries current processor telemetry without advancing the clock.
- `QUIT` -- Signals the server that this UI client is disconnecting.

2. **Core to UI State Broadcast (Port 5000):**

Core emits an atomic, pipe-delimited key-value string:

```
STATE|PC=0002|A=FE|B=00|CY=false|AC=false|OV=false|P=true|SP=07|R0=00|R1=00|R2=00|R3=00|R4=00|R5=00|R6
 =00|R7=00|INSTR=MOV A, #data|OPCODE=74|FETCH=PC=0x0000 -> Opcode 0x74|DECODE=MOV A, #data [0x74]|EXEC
 UTE=A : 00 -> FE|Q_SIZE=0|Q_CAP=16|Q_FRONT=0|Q_REAR=0|Q_ELEMS=|RAM=0000...
```

Termination Markers:

- `HALTED` -- Emitted when execution stops due to `0xFF` (HALT).
- `RESET_DONE` -- Emitted when reset or program reload is complete.

3. **Core to Logger Telemetry Stream (Port 5001):**

```
LOG|EXEC|CPU|PC=0002|A=FE|CY=false
LOG|INFO|CORE|CoreProcess initialized with default demo bytecode (13 bytes).
LOG|RESET|CORE|CPU and data structures reset.
LOG|LOAD|CORE|Loaded 9 bytes of custom bytecode.
LOG|SHUTDOWN|CORE|CoreProcess closing logger connection.
```

---

2. Team Member Responsibilities & Process Breakdown

Student Name	Student ID	Designated Component	Primary Source Files	Network Role
Shashidhara	25190148	Student 1 -- UI Process	`src/UIProcess.java` `src/SimulatorUI.java`	HTTP Server (8080/8000) TCP Client -> Port 5000

Chrisel Lobo	25190110	Student 2 -- Core Process	`src/CoreProcess.java`
			`src/CPU.java`
			`src/Memory.java`
			`src/Registers.java`
			`src/PSW.java`
			`src/DataStructures.java`
				TCP Server (Port 5000)
				TCP Client -> Port 5001

Arnold Noah	25190106	Student 3 -- Logging Process	`src/LoggerProcess.java` `simulator.log`	TCP Server (Port 5001) Disk File Sink

Cyrus Shobith Pinto	25190111	Team Leader -- Integration	`tests/TestCPU.java`
			`programs/QueueTest.java`
			`HOWTORUN.md`
			Architecture & Status Docs
				System Integrator & QA Lead

3. Core Process & CPU Execution Engine (Chrisel Lobo)

3.1 Architecture of `CoreProcess.java`

`CoreProcess` acts as the computational engine of the simulator.

- **Concurrency Model:** Uses a cached thread pool (`Executors.newCachedThreadPool()`) to handle concurrent inbound UI connections.
- **Thread Safety:** Every CPU state mutation is synchronized across an internal lock object (`synchronized (cpuLock)`). Even if both the Web Dashboard and the Swing GUI issue commands simultaneously, race conditions on registers and memory are mathematically prevented.
- **Resilient Logging Fallback:** When Core starts, it attempts to connect to Logger on `127.0.0.1:5001`. If Logger is not running, Core catches `IOException`, prints a console warning, falls back to standard output logging, and continues running without crashing.
- **Bytecode Sanitizer:** When a user inputs raw hex code in the UI (e.g. `74 42`), `parseHex()` strips delimiters, pairs nibbles, and verifies whether the final opcode is `0xFF` (`HALT`). If missing, it automatically appends `0xFF` so execution cannot walk off into uninitialized memory.

3.2 The FETCH -> DECODE -> EXECUTE Engine (`CPU.java`)

```
flowchart TD
    Start([CPU Step Invocation]) --> CheckHalt{Is CPU Halted?\nisHalt == true}
    CheckHalt -- YES --> Return([Return / No Action])
    CheckHalt -- NO --> Fetch[1. FETCH<br/>currOpcode = mem.readCode(pc)<br/>fetchPC = pc<br/>pc++]
    Fetch --> Decode[2. DECODE<br/>Lookup opcode in switch table<br/>Identify mnemonic, operands, format]
    Decode --> Execute[3. EXECUTE<br/>Fetch immediate operand if needed<br/>Perform ALU operation or Jump
    <br/>Update Registers & Flags in PSW<br/>Log state change string]
    Execute --> End([Step Complete / Emit State])
```

Step Method Walkthrough:

```
public void step() {
    if (isHalt) return;
    fetch();
    decode();
    execute();
}
```

1. `fetch()` **Method:**

```
   int fetchPC = pc;
   currOpcode = mem.readCode(pc);
   pc++;
   fetchLog = "PC=0x" + toHex(fetchPC, 4) + " -> Opcode 0x" + toHex(currOpcode, 2);
```

Retrieves the 8-bit instruction at `codeMemory[pc]`, increments the 16-bit Program Counter, and generates the string for the UI pipeline trace.

2. `decode()` **Method:**

Translates raw machine code into symbolic operations:

- `0x00` -> `NOP` (No Operation)
- `0x74` -> `MOV A, #data` (Immediate data transfer)
- `0x24` -> `ADD A, #data` (Immediate arithmetic addition)
- `0x04` -> `INC A` (Accumulator increment)
- `0x54` -> `ANL A, #data` (Logical bitwise AND)
- `0x80` -> `SJMP rel` (Relative jump)
- `0xE0` -> `ENQUEUE A` (Queue insertion)
- `0xD0` -> `DEQUEUE A` (Queue removal)
- `0xFF` -> `HALT` (Execution termination)

3. `execute()` **Method:**

Executes the target instruction and records state changes.

3.3 Complete Instruction Set Specification

1. `0x00` -- `NOP` **(No Operation)**
   - **Operation:** Advances PC by 1. Does not alter registers, flags, or memory.
   - **Code:** `statechangelog = "NOP";`
2. `0x74` -- `MOV A, #data` **(Move Immediate to Accumulator)**
   - **Byte sequence:** `0x74 [data]` (2 bytes)
   - **Operation:** Fetches immediate operand from code memory and stores it in Accumulator `A`.
   - **Flags:** Computes Parity (`P`). Does not alter Carry or Overflow.
3. `0x24` -- `ADD A, #data` **(Add Immediate with Flags)**
   - **Byte sequence:** `0x24 [data]` (2 bytes)
   - **Operation:** Adds immediate operand to current value of `A`.
   - **ALU Logic:**

```
int before = regs.A;
int operand = fetchOper();
int result = before + operand;
psw.updateAfterAdd(before, operand, result);
updateAcc(result);
```

   - **Flags Affected:**
   - **Carry (`CY`):** Set if unsigned sum exceeds 255 (`result > 0xFF`).
   - **Auxiliary Carry (`AC`):** Set if carry occurred from bit 3 to bit 4 (`((before & 0xF) + (operand & 0xF)) > 0xF`).
   - **Overflow (`OV`):** Set if signed two's complement overflow occurs.
   - **Parity (`P`):** Set if set-bit count in `A` is odd.
4. `0x04` -- `INC A` **(Increment Accumulator)**
   - **Byte sequence:** `0x04` (1 byte)
   - **Operation:** `A = (A + 1) & 0xFF`.
   - **Wraparound Rule:** Increments `0xFF` (255) to `0x00` (0).
   - **Flags Affected:** Updates Parity (`P`). **Does NOT affect Carry (`CY`)**, adhering to official 8051 hardware specification.
5. `0x54` -- `ANL A, #data` **(Bitwise AND with Accumulator)**
   - **Byte sequence:** `0x54 [data]` (2 bytes)
   - **Operation:** `A = A & fetchOper()`.
   - **Flags Affected:** Updates Parity (`P`). Clears Carry/Overflow.
6. `0x80` -- `SJMP rel` **(Short Jump Relative)**
   - **Byte sequence:** `0x80 [offset]` (2 bytes)
   - **Operation:** Unconditional relative jump. The target address is computed by adding the signed 8-bit offset to the updated PC.
   - **Two's Complement Signed Offset Math:**

```
int offset = fetchOper();
if (offset > 127) offset -= 256; // Converts unsigned 128..255 to negative -128..-1
pc += offset;
```

7. `0xE0` -- `ENQUEUE A` **(Enqueue Accumulator into FIFO Buffer)**
   - **Byte sequence:** `0xE0` (1 byte)
   - **Operation:** Writes value of `A` into circular queue at `qRear`, increments `qRear` modulo 16, increments `qSize`.
   - **Exception:** Throws `RuntimeException("Circular Queue Overflow")` if `qSize == 16`.
8. `0xD0` -- `DEQUEUE A` **(Dequeue into Accumulator)**
   - **Byte sequence:** `0xD0` (1 byte)
   - **Operation:** Reads value from `qFront` into Accumulator `A`, increments `qFront` modulo 16, decrements `qSize`.
   - **Exception:** Throws `RuntimeException("Circular Queue Underflow")` if `qSize == 0`.
9. `0xFF` -- `HALT` **(Halt Processor)**
   - **Byte sequence:** `0xFF` (1 byte)
   - **Operation:** Sets `isHalt = true`. Subsequent `step()` calls return immediately.

3.4 Data Structures: Hardware Stack & Circular Queue (`DataStructures.java`)

The Hardware Stack:

```
flowchart TD
    subgraph PUSH ["PUSH Operation"]
        p1([Start PUSH val]) --> p2["Increment Stack Pointer: sp++"]
        p2 --> p3{Is sp >= stack.length?}
        p3 -- YES --> p4[Throw Stack Overflow Exception]
        p3 -- NO --> p5["Write stack[sp] = val & 0xFF"]
        p5 --> p6([End PUSH])
    end

    subgraph POP ["POP Operation"]
        q1([Start POP]) --> q2{Is sp < 0x07?}
        q2 -- YES --> q3[Throw Stack Underflow Exception]
        q2 -- NO --> q4["Read val = stack[sp]"]
        q4 --> q5["Decrement Stack Pointer: sp--"]
        q5 --> q6([Return val])
    end
```

- **Storage:** 256-integer array (`stack = new int[256]`).
- **SP Initialization:** `sp = 0x07`.
- **Hardware Rule:** 8051 uses pre-increment on push and post-decrement on pop.

The Circular FIFO Queue:

```
flowchart TD
    subgraph ENQUEUE ["ENQUEUE Operation"]
        e1([Start ENQUEUE val]) --> e2{Is Queue Full?\nqSize == qCapacity}
        e2 -- YES --> e3[Throw Circular Queue Overflow Exception]
        e2 -- NO --> e4["Write queue[qRear] = val & 0xFF"]
        e4 --> e5["Advance qRear = (qRear + 1) % qCapacity"]
        e5 --> e6["Increment qSize++"]
        e6 --> e7([End ENQUEUE])
    end

    subgraph DEQUEUE ["DEQUEUE Operation"]
        d1([Start DEQUEUE]) --> d2{Is Queue Empty?\nqSize == 0}
        d2 -- YES --> d3[Throw Circular Queue Underflow Exception]
        d2 -- NO --> d4["Read val = queue[qFront]"]
        d4 --> d5["Advance qFront = (qFront + 1) % qCapacity"]
        d5 --> d6["Decrement qSize--"]
        d6 --> d7([Return val into Accumulator A])
    end
```

- **Storage:** 16-element buffer (`queue = new int[16]`).
- **Indices:** `qFront` (read pointer), `qRear` (write pointer), `qSize` (active count).
- **Modulo Arithmetic:** Prevents memory creep by recycling vacated buffer slots in $O(1)$ time complexity without shifting elements.

3.5 Memory Architecture (`Memory.java`)

- `codeMemory = new byte[8192]`: Stores bytecode instructions.
- `internalRAM = new byte[256]`: Stores general-purpose variables, register banks, and the stack.
- `loadProgram(byte[] prog)`: Clears program memory to `0x00` and copies bytecode starting at index 0.
- `dumpRAM(start, length)`: Formats RAM contents into hex strings for the real-time inspector.

---

4. UI Process & Visualization Subsystem (Shashidhara)

4.1 Architecture of `UIProcess.java`

`UIProcess` provides two synchronized interaction interfaces:

1. **Embedded HTTP Web Server:** Binds to `0.0.0.0:8080` (with secondary binding on `8000`), serving an interactive dark-mode dashboard with real-time DOM updates via JavaScript `fetch()`.
2. **Interactive Terminal CLI:** Runs concurrently on standard input (`System.in`), allowing users to type commands (`STEP`, `RUN`, `RESET`, `LOAD`, `QUIT`) directly in the shell.

4.2 The 6-Panel Real-Time Dashboard

The web interface is divided into six functional panels:

1. **Assembly / Opcode Editor:** Allows developers to input raw bytecode strings and click `[Load Hex / Code]`.
2. **CPU Execution Trace:** Displays the live status of the 3-stage pipeline:
   - Phase 1: `FETCH [ PC=0x0002 -> Opcode 0x24 ]`
   - Phase 2: `DECODE [ ADD A, #data [0x24] ]`
   - Phase 3: `EXECUTE [ A : FE -> 01 (CY=true OV=false) ]`
   - State transition: `ACC: 0xFE -> 0x01`, `CY: false -> true`.
3. **Registers & Status Word (`PSW`):** Real-time hexadecimal display of `PC`, `SP`, `ACC`, `B`, and general registers `R0` through `R7`. Flag badges dynamically illuminate when active: `[CY: 1]`, `[AC: 1]`, `[OV: 0]`, `[P: 1]`.
4. **Hardware Stack:** Tracks `SP`, shows calculated stack depth (`max(0, SP - 7)` pushed items), and dynamically dereferences the Top of Stack value: `RAM[SP] = 0xAA`.
5. **Circular FIFO Queue:** Visualizes `qSize` / `qCapacity`, pointer indices (`qFront`, `qRear`), and the array of enqueued items: `[ 0x02, 0x09 ]`.
6. **256-Byte Memory RAM Inspector:** Complete 16x16 hex dump of internal data RAM (`0x00` to `0xFF`).

4.3 Desktop Swing GUI (`SimulatorUI.java`)

- **Thread Decoupling:** Uses a dedicated background thread (`UI-ListenerThread`) to listen for incoming TCP packets from Core without freezing the user interface.
- **Event Dispatch Thread (EDT) Safety:** State updates are dispatched to the GUI thread using `SwingUtilities.invokeLater()` to prevent deadlocks and race conditions.
- **Animated Pipeline Indicators:** When a step executes, visual indicators for **FETCH**, **DECODE**, and **EXECUTE** illuminate sequentially in green with microsecond timer delays.

---

5. Logging Process & Audit Subsystem (Arnold Noah)

5.1 Architecture of `LoggerProcess.java`

- **Independent Process Daemon:** Executes in a dedicated JVM on **TCP port 5001**.
- **Continuous Multi-Session Acceptance Loop:**

```
while (running) {
    Socket coreSocket = server.accept();
    // Handle logging stream until Core disconnects...
}
```

Survives Core restarts and re-enters listening mode automatically.

- **Dual Destination Output:** Every log line is simultaneously written to standard output (`stdout`) and appended to `simulator.log` on disk.
- **Immediate Buffer Flush:** Executes `fileWriter.flush()` immediately after each record, ensuring that if power is lost or the process is killed, zero audit records are lost.

5.2 Structured Log Format

```
[Timestamp]              [Level] [Category  ] Message Details
[2026-09-29 09:20:43.707] [INFO ] [CORE      ] CoreProcess initialized with default demo bytecode (13 byt
    es).
[2026-09-29 09:21:07.389] [EXEC ] [CPU       ] PC=0002|A=FE|CY=false
[2026-09-29 09:21:46.236] [EXEC ] [CPU       ] PC=0004|A=01|CY=true
[2026-09-29 09:21:54.044] [EXEC ] [CPU       ] PC=0005|A=02|CY=true
[2026-09-29 09:22:08.389] [RESET] [CORE      ] CPU and data structures reset.
[2026-09-29 09:22:30.549] [LOAD ] [CORE      ] Loaded 3 bytes of custom bytecode.
```

6. System Integration, Testing & Verification (Cyrus Shobith Pinto)

6.1 Integration Challenges & Solutions

1. **Branch Merge Reconciliation:** Merged diverging branches (`origin/chrisel-core`, `origin/shashidhara-ui`, `origin/arnold-log`) into `main`. Standardized all modules into the default package to enable clean compilation via `javac -cp src src/*.java`.
2. **Lifecycle Mismatch:** Shashidhara's web dashboard makes short-lived HTTP connections, whereas Chrisel's Core originally exited upon reading EOF from the first client. Cyrus converted Core into a multi-threaded server using `ExecutorService` and `ServerSocket.accept()`, allowing Core to remain online permanently.
3. **Instruction Fall-off Protection:** When custom bytecode lacks a trailing `0xFF` (`HALT`), Cyrus implemented `NOP` (`0x00`) in `CPU.java` and automated `0xFF` padding in `CoreProcess.parseHex()`.
4. **Unified State Telemetry:** Standardized the `STATE|...` wire string so that both the 6-panel web dashboard and the Swing GUI receive all required telemetry in a single atomic transmission.

6.2 Instruction Test Suite Walkthrough (`tests/TestCPU.java`)

Test ID	Instruction Under Test	Input Conditions	Expected Outcome	Verification Metric
TC-01	`MOV A, #data`	Opcode `0x74 0x42`	`A = 0x42`	Verifies immediate data loading into ACC.
TC-02	`MOV A, #0x00`	Opcode `0x74 0x00`	`A = 0x00`	Verifies edge-case zero loading.
TC-03	`ADD A, #data` (No Carry)	Initial `A = 0x10`, ADD `0x05`	`A = 0x15`, `CY = false`	Verifies basic addition without overflow.
TC-04	`ADD A, #data` (Carry)	Initial `A = 0xFE` (254), ADD `0x03`	`A = 0x01`, `CY = true`	Verifies 8-bit overflow (257) sets Carry.
TC-05	`INC A`	Initial `A = 0x07`	`A = 0x08`	Verifies accumulator increment.
TC-06	`INC A` (Wraparound)	Initial `A = 0xFF` (255)	`A = 0x00`	Verifies 8-bit wraparound (255 -> 0).
TC-07	`ANL A, #data`	Initial `A = 0xAB`, ANL `0x0F`	`A = 0x0B`	Verifies logical bitwise AND masking.
TC-08	`SJMP rel` (Forward Jump)	Opcode `0x80 0x01`, followed by `INC A`	`A = 0x00` (INC skipped)	Verifies relative jump over instructions.
TC-09	`ENQUEUE A`	`A = 0x55`, Opcode `0xE0`	`qSize = 1`	Verifies queue insertion.
TC-10	`DEQUEUE A`	Enqueue `0x33`, clear `A=0`, Dequeue	`A = 0x33`	Verifies queue retrieval into ACC.
TC-11	`HALT`	Opcode `0xFF`	`isHalt = true`	Verifies processor halt state.
TC-12	Full Demo: Dequeue FIFO	Multi-step program	`A = 0x02`	Verifies FIFO ordering on full pipeline.
TC-13	Full Demo: Queue Size	Multi-step program	`qSize = 1`	Verifies queue remaining count.
TC-14	Full Demo: Carry State	Multi-step program	`CY = true`	Verifies carry persists through pipeline.

Result: `Results: 14 PASSED, 0 FAILED out of 14 tests -- ALL TESTS PASSED [PASS]`

6.3 Queue FIFO Validation Runner (`programs/QueueTest.java`)

Executes an assembly validation routine:

```
MOV A, #10    ; A = 10
ENQUEUE A     ; Queue: [10]
MOV A, #30    ; A = 30
ENQUEUE A     ; Queue: [10, 30]
MOV A, #5     ; A = 5
ENQUEUE A     ; Queue: [10, 30, 5]
DEQUEUE A     ; Expected A = 10 (First-in, first-out) -> PASS
DEQUEUE A     ; Expected A = 30 -> PASS
DEQUEUE A     ; Expected A = 5  -> PASS
HALT
```

Result: `3/3 dequeue operations verified FIFO order -- PASSED [PASS]`

6.4 Standalone vs. Multi-Process Performance Benchmark

Benchmark Metric	Standalone (Single Process)	Multi-Process (3 Processes + TCP IPC)	Technical Analysis
Step Execution Latency	$< 0.05 \text{ ms}$	$1.2 - 2.8 \text{ ms}$	IPC overhead due to TCP socket serialization and loopback network stack.
Total Demo Execution	$\approx 0.1 \text{ ms}$	$25 - 45 \text{ ms}$	Synchronous round-trip socket communication for each instruction.
Total Memory Allocated	$\approx 48 \text{ MB}$ (1 JVM)	$\approx 145 \text{ MB}$ (3 JVMs)	Overhead of 3 independent JVM heaps and runtimes.
CPU Core Scheduling	Single Core Bound	Multi-Core Concurrent	OS scheduler distributes UI, Core, and Logger across separate hardware cores.
System Reliability	Zero Fault Isolation	Total Process Isolation	UI crash leaves CPU intact; Core crash leaves Logger audit intact.

7. The Hostile Examiner's Viva Defense (Question & Answer Drills)

Q1: "Show me where your processes share memory."

Answer: *"They do not share memory. Each component runs in an independent operating system process with its own private JVM heap and address space. All communication is strictly achieved through message-passing over TCP loopback sockets on ports 5000 and 5001."*

Q2: "Why does the 8051 Stack Pointer reset to `0x07` instead of `0x00`?"

Answer: *"In the STC89C52 internal RAM map, addresses `0x00` through `0x07` are physically assigned to Register Bank 0 (`R0-R7`). Because the 8051 stack uses pre-increment on push, setting SP to `0x07` ensures the first pushed byte is written to address `0x08` (Bank 1), preventing the stack from overwriting active working registers."*

Q3: "Why did you implement a Circular Queue instead of a standard linear queue?"

Answer: *"A linear queue in fixed hardware memory experiences false overflow once `qRear` hits the end of the array, even if elements have been dequeued from the front. A circular queue uses modulo indexing (`(index + 1) % capacity`) to recycle vacated memory slots, providing $O(1)$ constant time operations without moving elements in memory."*

Q4: "What happens if LoggerProcess is offline when CoreProcess starts?"

Answer: *"In `CoreProcess.connectLogger()`, the connection attempt is wrapped in a `try-catch` block. If Logger is offline, Core catches the `IOException`, prints a console notification, falls back to standard output logging, and continues operating without throwing an unhandled exception."*

Q5: "Why did you use bitwise masking `& 0xFF` on all register operations?"

Answer: *"Java does not possess an unsigned 8-bit primitive; Java `byte` is signed (-128 to 127) and `int` is 32-bit signed. Without `& 0xFF`, values greater than 127 sign-extend into negative 32-bit integers. Masking with `0xFF` constrains every variable strictly to the range `0x00-0xFF`, accurately replicating physical 8-bit hardware registers."*

Q6: "How does your simulator calculate the Auxiliary Carry (AC) flag?"

Answer: *"Auxiliary carry represents a carry out of bit 3 into bit 4 (the lower nibble). It is calculated in `PSW.java` using: `((operand1 & 0x0F) + (operand2 & 0x0F)) > 0x0F` This flag is required for BCD (Binary Coded Decimal) arithmetic instructions like `DA A`."*

Q7: "How does the simulator calculate the Overflow (OV) flag?"

Answer: *"Overflow indicates signed two's complement overflow (-128 to +127). It occurs if adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result: `boolean sign1 = (operand1 & 0x80) != 0; boolean sign2 = (operand2 & 0x80) != 0; boolean signR = ((result & 0xFF) & 0x80) != 0; ov = (sign1 == sign2) && (sign1 != signR);`"*

Q8: "How does your SJMP instruction handle backward jumps?"

Answer: *"In 8051 assembly, the SJMP relative offset is an 8-bit signed two's complement number (-128 to +127). When read from memory, a value like* `0xFE` *appears as positive 254. In* `CPU.java`*, we evaluate:* `if (offset > 127) offset -= 256; pc += offset;` *This converts 254 into* `-2`*, correctly branching backward."*

Q9: "Why is opcode `0x00` implemented as NOP instead of throwing an unknown instruction error?"

Answer: *"In the official 8051 instruction set,* `0x00` *is the standard opcode for* `NOP` *(No Operation). It simply increments PC by 1 without altering registers or memory. Treating it as an unknown instruction would cause the simulator to crash when encountering valid NOP padding."*

Q10: "What happens if a user submits custom hex in the UI that lacks a HALT (`0xFF`) opcode?"

Answer: *"In* `CoreProcess.parseHex()`*, the byte array is inspected. If the final byte is not* `0xFF`*, Core automatically allocates an extra byte and appends* `0xFF` *to the program. This ensures the CPU halts safely rather than running into uninitialized memory."*

Q11: "How do you guarantee thread safety between UI socket requests and CPU execution in CoreProcess?"

Answer: *"In* `CoreProcess.java`*, all CPU mutations (*`step()`*,* `initialize()`*,* `loadProgram()`*) are synchronized across an explicit monitor lock (*`synchronized (cpuLock)`*). While one thread executes a step or program reload, all other incoming socket requests wait, ensuring atomic state transitions."*

Q12: "How does `SimulatorUI.java` prevent Swing thread violations?"

Answer: *"Swing components must only be modified on the Event Dispatch Thread (EDT). When* `SimulatorUI` *receives a* `STATE|...` *message on its background socket listener thread (*`UI-ListenerThread`*), it wraps the DOM update in* `SwingUtilities.invokeLater(() -> handleCoreMessage(msg))`*."*

Q13: "What happens if a UI client disconnects unexpectedly?"

Answer: *"In* `CoreProcess.java`*, each client is handled in a* `ClientHandler` *thread. If the client disconnects,* `readLine()` *returns* `null` *or throws an* `IOException`*, which closes only that client socket. The main* `ServerSocket` *loop continues running and accepting new connections."*

Q14: "Why did you choose TCP sockets over named pipes or shared memory?"

Answer: *"TCP loopback sockets provide a portable, POSIX-compliant, bi-directional, reliable byte stream that works natively in standard Java across Windows, Linux, and macOS without requiring native C code, JNI, or platform-specific dependencies."*

Q15: "What are the exact ports used by the system?"

> ***Answer: Port 5001:*** *CoreProcess connects as a client to LoggerProcess (inbound audit telemetry).* ***Port 5000:*** *UIProcess connects as a client to CoreProcess (directives & state broadcast). * **Port 8080 (and 8000):** Browser connects to UIProcess (HTTP dashboard SPA)."*

---

8. Step-by-Step Practical Demonstration Procedure

1. **Terminal 1 -- Logging Process:**

```
cd c:\Users\Cyrus\Downloads\!pbl\STC89C52-Simulator
java -cp src LoggerProcess
```

2. **Terminal 2 -- Core Execution Process:**

```
cd c:\Users\Cyrus\Downloads\!pbl\STC89C52-Simulator
java -cp src CoreProcess
```

3. **Terminal 3 -- UI Process:**

```
cd c:\Users\Cyrus\Downloads\!pbl\STC89C52-Simulator
java -cp src UIProcess
```

4. **Browser Verification:**

Navigate to `http://localhost:8080` (or `http://localhost:8000`).

- Click **Step** to observe live Fetch -> Decode -> Execute pipeline transitions.
- Click **Run** to execute to `HALTED`.
- Verify timestamped audit logs in Terminal 1 and in `simulator.log`.

5. **Automated Verification:**

```
java -cp "src;tests" TestCPU
java -cp "src;programs" QueueTest
```